

Informe Sanos y Salvos

Integrantes: Alexis Ramirez
Felipe Salazar
Matias Vega

I. Índice

I. Índice.....	2
II. Contexto del Proyecto.....	3
• Descripción del Problema:.....	3
• Solución Propuesta:.....	4
• Impacto:.....	4
III. Definición de Requerimientos.....	5
• Requerimientos Funcionales (RF):.....	5
• Requerimientos no Funcionales (RNF):.....	5
IV. Historias de Usuario (H.U.).....	6
V. Arquitectura.....	7
• Backend:.....	7
• Frontend:.....	8
• Método de Branch:.....	8
VI. Tecnologías Utilizadas / Justificación Técnica.....	9
• Estrategia de despliegue (Backend):.....	9
• Estrategia de despliegue (Frontend):.....	10
VII. Justificación.....	11
• Lenguaje (Backend):.....	11
• Base de datos y Persistencia:.....	11
• Ecosistema de Microservicios y API Management:.....	12
• Capa de Presentación (Frontend):.....	12
• Contenerización, Orquestación e Infraestructura:.....	13
VIII. Revisión Ética.....	14
• Seguridad:.....	14
• Privacidad:.....	14
• Sostenibilidad:.....	14
IX. Conclusión.....	15
X. Diagramas.....	16
1. Diagrama de Componentes: (No se si va).....	16
2. (Diagrama Completo con cluster de kubernetes y docker).....	16
3. Diagrama Arquitectura (ms-Auth).....	16
4. Diagrama Arquitectura (ms-User).....	16
5. Diagrama Arquitectura (ms-Pet).....	16
6. Diagrama Arquitectura (ms-Notification).....	16
7. Diagrama Arquitectura (BFF).....	16
8. Diagrama Arquitectura (Api Gateway).....	16
9. Diagrama Arquitectura (Frontend (No se si va)).....	16

II. Contexto del Proyecto

- **Descripción del Problema:**

En la actualidad, el aumento en la tenencia de mascotas ha hecho que su seguridad sea una prioridad, sin embargo, los sistemas de recuperación ante extravíos son deficientes. Cuando una mascota se pierde, la información se dispersa de forma caótica en publicaciones de redes sociales, carteles de papel en las calles y reportes aislados en clínicas veterinarias o refugios.

Esta fragmentación genera diversos problemas críticos:

- **Información Desestructurada:** Los reportes actuales suelen ser simples formularios web o correos que carecen de datos técnicos estandarizados, lo que dificulta el cruce de información.
- **Falta de Coordinación:** No existe un puente de comunicación efectivo entre las municipalidades, las organizaciones animalistas y veterinarias que reciben animales sin identificación.
- **Invisibilidad Geográfica:** Es extremadamente difícil visualizar zonas con mayor incidencia de extravíos, lo que impide tomar acciones preventivas en sectores específicos.
- **Ineficiencia Temporal:** Los métodos manuales son lentos y, en el caso de mascotas perdidas, las primeras horas son vitales; la demora actual reduce drásticamente las probabilidades de éxito.

● Solución Propuesta:

La plataforma **Sanos y Salvos** propone una solución moderna basada en una arquitectura de microservicios para centralizar y automatizar todo el ecosistema de búsqueda. La propuesta no es solo una base de datos, sino un núcleo inteligente que conecta a todos los actores relevantes (**dueños, Clínicas, Municipios y Refugios**).

Los Pilares de la propuesta técnica incluyen:

- **Centralización de Datos:** Unificar los reportes estructurados que provienen de distintas fuentes (Dueños, Personas que encuentran animales y entidades oficiales) en un solo lugar.
- **Geolocalización:** Un sistema que permite visualizar en un mapa interactivo los reportes, facilitando la identificación de patrones de extravío y ayudando a los rescatistas a filtrar cercanía.
- **Escalabilidad y Modularización:** Gracias al diseño de microservicios, el sistema puede crecer de forma independiente, permitiendo que el módulo usuarios por ejemplo, pueda escalar de forma independiente soportando más usuarios simultáneamente sin degradar el rendimiento general.

● Impacto:

La implementación de esta plataforma busca transformar la gestión de bienestar animal en las ciudades:

- **Reducción en Tiempos de Respuesta:** Al generalizar los formularios de extravío de las mascotas, se espera que el tiempo entre extravío y el reencuentro se reduzca significativamente, minimizando el trauma tanto para la mascota como para el dueño.
- **en la Tasa de Recuperación:** Al contactar directamente a las clínicas que reciben animales sin placa con reportes de sueños, se eliminan los puntos ciegos o retrasos del proceso actual.
- **Toma de Decisiones Basada en Datos:** Las municipalidades podrán utilizar la información geográfica para realizar campañas de microchip o tenencia responsable en las zonas donde se detecten más pérdidas.
- **Responsabilidad Ética y Técnica:** Al integrar principios éticos en el diseño, se garantiza que los datos sensibles de los ciudadanos estén protegidos, mientras se ofrece una herramienta socialmente responsable y sostenible en el tiempo.

III. Definición de Requerimientos

- **Requerimientos Funcionales (RF):**

- **RF 1 - Gestión Estructurada de Mascotas:** El sistema debe permitir el registro de mascotas incluyendo fotos, características físicas, ubicación y datos de contacto.
- **RF 2 - Visualización Geográfica:** La plataforma debe mostrar en un mapa interactivo los reportes activos para identificar zonas con alta incidencia de extravíos.
- **RF 3 - Búsqueda por Radio Geográfico y Filtros (Revisar este RF):** El sistema debe permitir a los usuarios buscar reportes de mascotas perdidas o encontradas aplicando filtros por área de cercanía (ej. a 1 km, 5 km o 10 km de su ubicación actual mediante PostGIS) y por atributos estáticos (especie, color principal, tamaño y fecha).
- **RF 4 - Interfaz de Usuario:** El sistema debe ofrecer una interfaz intuitiva y responsiva (Frontend) para facilitar el registro rápido en situaciones de emergencia.

- **Requerimientos no Funcionales (RNF):**

- **RNF 1 - Escalabilidad:** El sistema, al estar basado en microservicios, debe soportar el escalado horizontal automático para manejar picos de concurrencia (por ejemplo, búsquedas masivas tras desastres naturales o eventos con pirotecnia).
- **RNF 2 - Rendimiento:** Las consultas/ filtros de búsqueda debe retornar resultados en un tiempo de 15 a 20 segundos.
- **RNF 3 - Disponibilidad:** La plataforma debe operar bajo un acuerdo de nivel de servicio (SLA) de 99.9% de *uptime*, siendo crítica su disponibilidad durante emergencias en horarios no hábiles.
- **RNF 4 - Seguridad:** Toda comunicación entre el cliente y los servidores debe estar cifrada (HTTPS/TLS 1.3), y las contraseñas o datos de contacto deben almacenarse mediante algoritmos de *hashing* seguros (ej. bcrypt).

IV. Historias de Usuario (H.U.)

- **H.U. 1: Registro de Mascotas Perdidas.**
 - **Como:** Dueño de una mascota extraviada,
 - **Quiero:** subir una foto y la ubicación del último avistamiento al sistema,
 - **Para que:** las clínicas y refugios cercanos estén en alerta y el motor de coincidencias empiece a trabajar. (Revisar este)
- **H.U. 2: Reporte de animal encontrado.**
 - **Como:** veterinario que recibe un animal sin identificación,
 - **Quiero:** ingresar las características del animal en la plataforma,
 - **Para :** verificar si existe un reporte de pérdida que coincida con el ejemplar recibido.
- **H.U. 3: Notificación de Coincidencia.**
 - **Como:** usuario de la plataforma,
 - **Quiero:** recibir una alerta automática cuando el sistema detecte una mascota encontrada con rasgos similares a la mía,
 - **Para:** agilizar el proceso de recuperación y contacto.
- **H.U. 4: Alertas por Proximidad.**
 - **Como:** Usuario registrado en la plataforma,
 - **Quiero:** Configurar mi zona residencial o cuadrante de interés,
 - **Para que:** El sistema me envíe una notificación inmediata cada vez que se cree un nuevo reporte de mascota perdida o encontrada dentro de mi radio configurado.

V. Arquitectura

- **Backend:**

- El backend del proyecto está diseñado como una arquitectura de microservicios basada en Spring Boot. En el repositorio los servicios principales se encuentran bajo la carpeta Backend e incluyen, entre otros, api-gateway, bff, ms-auth, ms-base, ms-mascota, ms-notification y ms-usuario. Cada microservicio es una aplicación Spring Boot con responsabilidades delimitadas: api-gateway actúa como punto de entrada, enrutamiento y control de seguridad; bff (Back-For-Front) adapta y orquesta las respuestas para el frontend y maneja la lógica de tokens JWT; ms-auth se encarga de la autenticación y autorización (soporte OAuth2 / resource-server y validación); los servicios de dominio (ms-usuario, ms-mascota, ms-notification, ms-base) contienen la lógica de negocio, persistencia y notificaciones.
- El stack tecnológico principal del backend incluye Spring Boot 4, Spring Data JPA para persistencia, PostgreSQL como base de datos relacional y Flyway para migraciones. Para seguridad se utiliza Spring Security, con soporte para JWT y OAuth2 (ver dependencias en build.gradle). Algunos módulos incorporan capacidades reactivas y cache con Redis (spring-boot-starter-data-redis-reactive). Lombok se utiliza para reducir boilerplate y spring-boot-starter-validation para validación de entradas. Las pruebas se apoyan en los starters de testing de Spring Boot y JUnit Platform.
- La comunicación entre microservicios y con el frontend es por HTTP/REST con intercambio de JSON; el api-gateway y el bff gestionan el forwarding de solicitudes y la verificación/propagación de tokens de autenticación. Para despliegue existen Dockerfile en varios servicios y manifiestos Kubernetes en Backend/*/k8s/ (deployments, secrets, config), lo que permite ejecutar la arquitectura tanto con Docker Compose en desarrollo como en un clúster Kubernetes en producción. En desarrollo se utilizan dependencias relacionadas con Docker Compose para facilitar integración local.

- **Frontend:**

- El frontend es una aplicación web basada en Next.js (versión indicada en package.json) y React con TypeScript. La estructura del proyecto está en la carpeta frontend y sigue la convención de páginas y componentes de Next.js; los componentes reutilizables se encuentran en components, las rutas y vistas en app, hooks personalizados en hooks y utilidades en lib.
- Las librerías y herramientas destacadas usadas en el frontend incluyen TailwindCSS y PostCSS para estilos, Radix UI para componentes accesibles, react-hook-form y zod para manejo y validación de formularios, leaflet y react-leaflet para mapas, recharts para gráficos y sonner para notificaciones. También existen utilidades como clsx y class-variance-authority para manejo de clases, y next-themes para gestión de temas. El proyecto está configurado para build con next build y ejecución en producción con next start (scripts en package.json).
- El flujo de datos entre frontend y backend se realiza mediante llamadas HTTP a las APIs expuestas por el bff o directamente a través del api-gateway. El proceso de autenticación consiste en que el usuario envía credenciales desde las rutas de login o register del frontend, el bff/ms-auth valida y devuelve un JWT, y el frontend almacena y adjunta ese token en el encabezado Authorization en peticiones subsecuentes a endpoints protegidos. La aplicación está preparada para ser desplegada en entornos que soporten Node/Next.js y puede integrarse en contenedores Docker para CI/CD y despliegue en Kubernetes.

- **Método de Branch:**

Se crea una rama cada vez que se quiera hacer una actividad nueva, es decir por cada actividad a realizar rama creada específica para esa actividad

VI. Tecnologías Utilizadas / Justificación Técnica

- **Backend:** Se utilizará **Java 25 (LTS)** con **Spring Boot 4.0.5**, dada su robustez para ecosistemas empresariales y la facilidad para desplegar módulos independientes (ej. Servicio de Usuarios, Servicio de Mascotas, Servicio de notificaciones).
- **Frontend:** **React.js 18**, desarrollados estrictamente con **TypeScript 5** para dotar al proyecto de tipado estático, mejorando el mantenimiento y reduciendo errores en el tiempo de ejecución. Se integrarán librerías como **Leaflet 1.9** para la renderización de la visualización geográfica fluida solicitada en los requerimientos.
- **Base de Datos:** **PostgreSQL 16** con la extensión **PostGIS 3.4**. Es esencial para manejar consultas espaciales complejas

Infraestructura y Despliegue:

- **Estrategia de despliegue (Backend):**
 - **Arquitectura:** Microservicios orquestados (API Gateway + BFF + servicios independientes: ms-auth, ms-usuario, ms-mascota, ms-notification, ms-base, etc.) desplegados como contenedores con networking interno y servicios expuestos vía Gateway/Ingress. Persistencia separada (bases de datos por servicio donde aplique).
 - **Plataformas de Referencia:**
 - Docker (contenedorización)
 - Kubernetes (orquestación)
 - Docker Compose (entorno de desarrollo local)
 - Traefik/Ingress (enrutamiento y TLS en cluster).
 - **Descripción:** Cada microservicio se empaqueta en una imagen Docker (multi-stage builds para optimizar tamaño). Para desarrollo local se usa docker-compose para levantar dependencias (DB, servicios) rápidamente; para pruebas de integración y simulación de producción se despliega en un clúster Kubernetes local (minikube/kind) usando manifests existentes (Deployment, Service, Secret, ConfigMap, Postgres manifest para ms-auth). El API Gateway y BFF centralizan rutas y autenticación; los servicios se comunican por HTTP interno o mensajería si se añade. El despliegue en Kubernetes incluye réplicas, probes (liveness/readiness), ConfigMaps/Secrets y políticas de recursos.

- Justificación Técnica: Contenerizar garantiza entornos reproducibles y CI/CD consistente; Kubernetes aporta orquestación, auto-recuperación y scaling (réplicas, HPA) necesarios para simular producción y validar tolerancia a fallos. Separar desarrollo (Compose) y staging/production (K8s) acelera iteración mientras se comprueban comportamientos reales de red y despliegue. El uso de secrets y manifests específicos (p. ej. Postgres) facilita gestión segura de credenciales y configuración.

● Estrategia de despliegue (Frontend):

- Arquitectura: Contenerización de la aplicación Next.js y entrega mediante un servidor web interno (Nginx) dentro del contenedor; orquestación en cluster local para simular producción y gestionar réplicas. Se apoya en un Ingress (Traefik) para enrutado, TLS y balanceo.
- Plataformas de Referencia:
 - Docker (construcción y empaquetado)
 - Kubernetes (orquestación, réplicas, rolling updates)
 - Nginx (servidor web para servir assets estáticos y cacheo)
 - Traefik/Ingress (entrada al cluster).
- Descripción: La imagen frontend se genera con build optimizado (multi-stage: build de Next.js → export/static o build + Node server según estrategia) y luego se sirve con Nginx para archivos estáticos y caché de recursos. En Kubernetes se despliega como Deployment con Service y reglas de Ingress para exponerla; se aplican probes, límites/requests y configuración de caching/CORS en Nginx. Para desarrollo se puede usar pnpm/next dev o docker-compose que monte volúmenes para hot-reload.
- Justificación Técnica: Nginx ofrece alto rendimiento para servir assets estáticos, control de caché y encabezados (Gzip, Brotli), además de desacoplar la entrega estática del runtime de Node si se opta por export estático. Kubernetes permite simular condiciones de producción (réplicas, actualizaciones sin downtime, rollback) y gestionar recursos/red en un entorno cercano al real. Mantener el build separado del runtime (multi-stage) reduce la superficie de ataque y el tamaño de la imagen.

VII. Justificación

- **Lenguaje (Backend):**

Java 25 (LTS) & Spring Boot 4.0.5, java proporciona un tipado fuerte, excelente gestión de memoria y madurez en el ecosistema de backend. El uso de la versión LTS más reciente asegura soporte a largo plazo y acceso a características modernas de rendimiento (como los *Virtual Threads* para alta concurrencia). Spring Boot 4 simplifica drásticamente la configuración y el desarrollo de microservicios mediante su enfoque de "convención sobre configuración", permitiendo aislar la lógica de negocio en módulos independientes ms-auth, ms-pets, ms-notification, ms-atuh, etc.) que pueden escalar de forma autónoma.

- **Base de datos y Persistencia:**

- **PostgreSQL 16 & PostGIS 3.4:**

Al tratarse de una plataforma donde la ubicación de los avistamientos y extravíos es crítica, se descartó el almacenamiento relacional común o NoSQL plano. **PostGIS** transforma a PostgreSQL en una base de datos espacial robusta, permitiendo realizar consultas geográficas complejas (cálculo de distancias, radios de búsqueda, cuadrantes de interés) directamente mediante lenguaje SQL indexado, garantizando respuestas en milisegundos sin sobrecargar el servidor de aplicaciones.

- **Spring Data JPA & Flyway:** JPA abstrae la complejidad de las consultas SQL tradicionales y agiliza el desarrollo a través del mapeo objeto-relacional (ORM). Por su parte, **Flyway** es fundamental en una arquitectura distribuida para el control de versiones de la base de datos; automatiza y aplica las migraciones de esquemas en cada despliegue, asegurando que todos los microservicios y entornos (desarrollo, producción) estén perfectamente sincronizados.

- **Ecosistema de Microservicios y API Management:**

- **API Gateway & BFF (Backend For Frontend):**

El API Gateway centraliza el punto de entrada al clúster, gestiona la seguridad periférica y enruta las peticiones de forma eficiente. El patrón BFF optimiza la comunicación con el cliente web (Next.js), orquestando llamadas concurrentes a múltiples microservicios internos y unificando las respuestas para reducir la latencia y la sobrecarga de red en el navegador.

- **Spring Security & OAuth2 / JWT:**

Garantizan un entorno seguro mediante autenticación stateless utilizando tokens JWT. Esto permite que cada microservicio valide la identidad del usuario de manera independiente (Resource Server) sin necesidad de consultar constantemente a un servidor central de sesión.

- **OpenAPI / Swagger:**

Actúa como el contrato de comunicación oficial entre el Backend y el Frontend. Automatiza la generación de documentación viva y proporciona una interfaz interactiva para probar los endpoints en tiempo real, reduciendo errores de integración entre los desarrolladores y acelerando el ciclo de desarrollo.

- **Capa de Presentación (Frontend):**

- **Next.js, React 18 & TypeScript 5:**

Next.js ofrece una estructura de enrutamiento moderna basada en el sistema de archivos y capacidades de optimización nativas para renderizado híbrido. React 18 permite construir interfaces de usuario modulares y reactivas mediante componentes reutilizables. El uso estricto de TypeScript 5 añade tipado estático, eliminando por completo los errores comunes en tiempo de ejecución y facilitando el mantenimiento del código a gran escala.

- **Leaflet & React-Leaflet:**

Herramienta open-source clave para la renderización fluida del mapa interactivo. Al integrarse con los datos geométricos provistos por

PostGIS, permite trazar marcadores, perímetros y mapas de calor con un consumo mínimo de recursos en el cliente.

- **Contenerización, Orquestación e Infraestructura:**

- **Docker & Docker Compose:**

Docker garantiza el principio de "funciona en mi máquina y funciona en producción", empaquetando cada microservicio con sus dependencias exactas en imágenes ligeras y aisladas. Docker Compose es la pieza clave para el entorno de desarrollo local, permitiendo que el equipo levante todo el ecosistema (múltiples bases de datos, pasarelas y servicios) con un solo comando.

- **Kubernetes (K8s) & Traefik/Ingress:**

Para simular un entorno de producción real y de alta disponibilidad, se seleccionó Kubernetes. Este orquestador proporciona capacidades críticas como auto-recuperación escalado horizontal automático (HPA) ante eventos de alta concurrencia y actualizaciones sin tiempo de inactividad. Traefik/Ingress actúa como el controlador de tráfico del clúster, gestionando el balanceo de carga interno y la terminación segura de certificados TLS/HTTPS.

VIII. Revisión Ética

- **Seguridad:**

La plataforma implementará autenticación basada en JWT (JSON Web Tokens) y el protocolo OAuth2, garantizando un acceso seguro y sin estado (stateless).

- **Mejora ética para Mitigación de Riesgos:** Como medida clave de diseño, los microservicios internos nunca estarán expuestos directamente a internet. Se implementará un API Gateway como único punto de entrada. Esto oculta la topología interna del sistema, centraliza la validación de tokens y permite aplicar políticas restrictivas de Rate Limiting para evitar saturación o ataques de fuerza bruta.

- **Privacidad:**

El sistema se desarrollará bajo los principios de "Privacidad por Diseño", alineándose con las normativas locales de protección de datos y tomando como referencia el GDPR europeo.

- **Protección de Datos Sensibles:** La ubicación exacta de un usuario o rescatista nunca se expondrá públicamente en texto plano; el mapa aplicará un difuminado geográfico (mostrando un radio aproximado de cuadradas). Asimismo, los datos de contacto directos (teléfono, dirección) permanecerán ocultos y solo se revelarán con el consentimiento explícito de ambas partes tras verificarse una coincidencia en el sistema, mitigando riesgos de extorsión o suplantación.

- **Sostenibilidad:**

La infraestructura se diseñará para minimizar la huella de carbono digital y el consumo de recursos de los servidores.

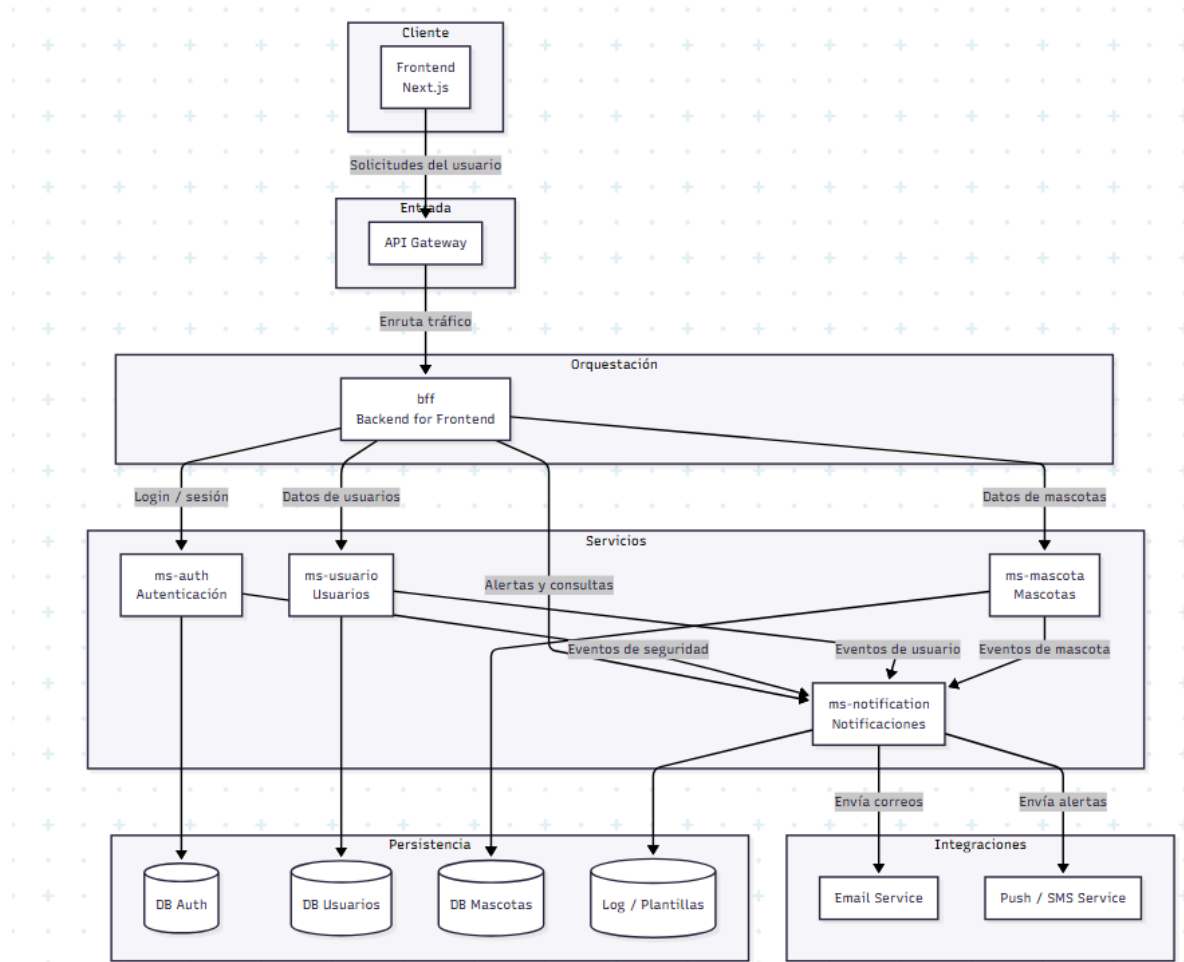
- **Optimización de Recursos:** Se utilizarán imágenes de contenedores ligeros (ej. Alpine Linux) para los microservicios en Java.
- **Eficiencia de Código:** A nivel de persistencia de datos, se aplicará una estricta optimización de consultas JPA/Hibernate (evitando el problema N+1, utilizando lazy loading y paginación en las búsquedas). Además, el uso de sistemas de caché (como Redis) para consultas frecuentes evitará sobrecargar la base de datos de manera innecesaria, reduciendo drásticamente los ciclos de CPU y, en consecuencia, el consumo energético del ecosistema en la nube.

IX. Conclusión

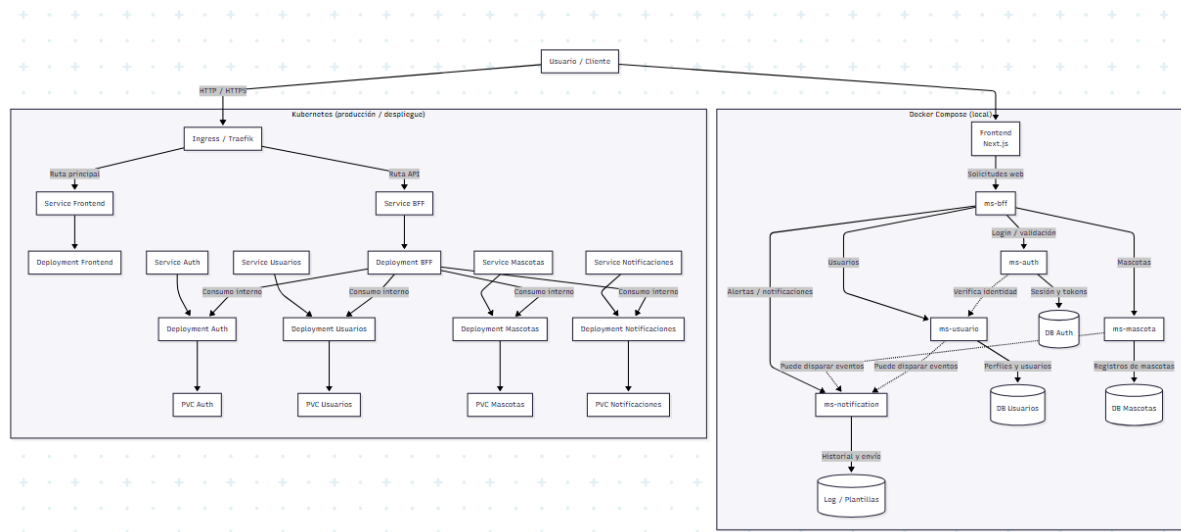
Este sistema busca mejorar y optimizar el rendimiento en rescates de mascotas logrando que las pérdidas de estas se reduzca dramáticamente una de las diversas formas de mejorar el proceso son a través de una página de web que centralice la búsqueda de mascotas mostrando un rango donde se perdió más notificaciones para los usuarios y/o clínicas veterinarias que apoyen al sistema donde estas serán informadas de una mascota perdida o encontrada para evitar búsquedas en zonas innecesarias, las notificaciones de las mascotas encontradas se llevarán a cabo después de realizar una comparación donde cada solicitud creada empezará a compararse por un filtro con mascotas con la mayor cantidad de rasgos para mostrarles las mas parecidas y en los casos de crear un reporte de una mascota encontrada de otra persona este “filtro” se vuelve a aplicar para mostrar las mascotas con solicitud de búsqueda el objetivo de esto es poder agilizar los reportes de mascotas perdidas y encontradas con el objetivo de evitar mucho reportes duplicados

X. Diagramas

1. Diagrama de Componentes:

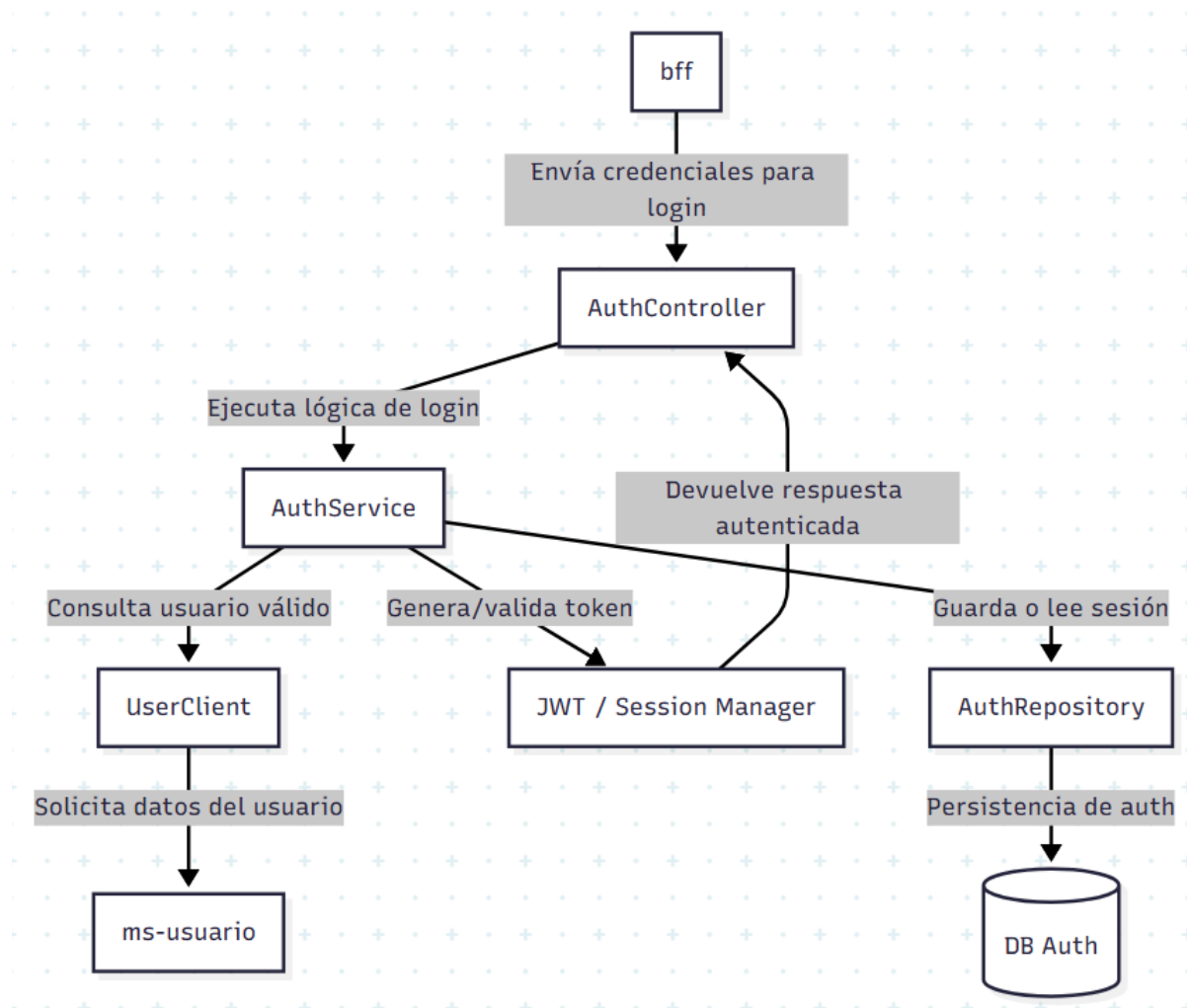


2. (Diagrama Completo con cluster de kubernetes y docker)

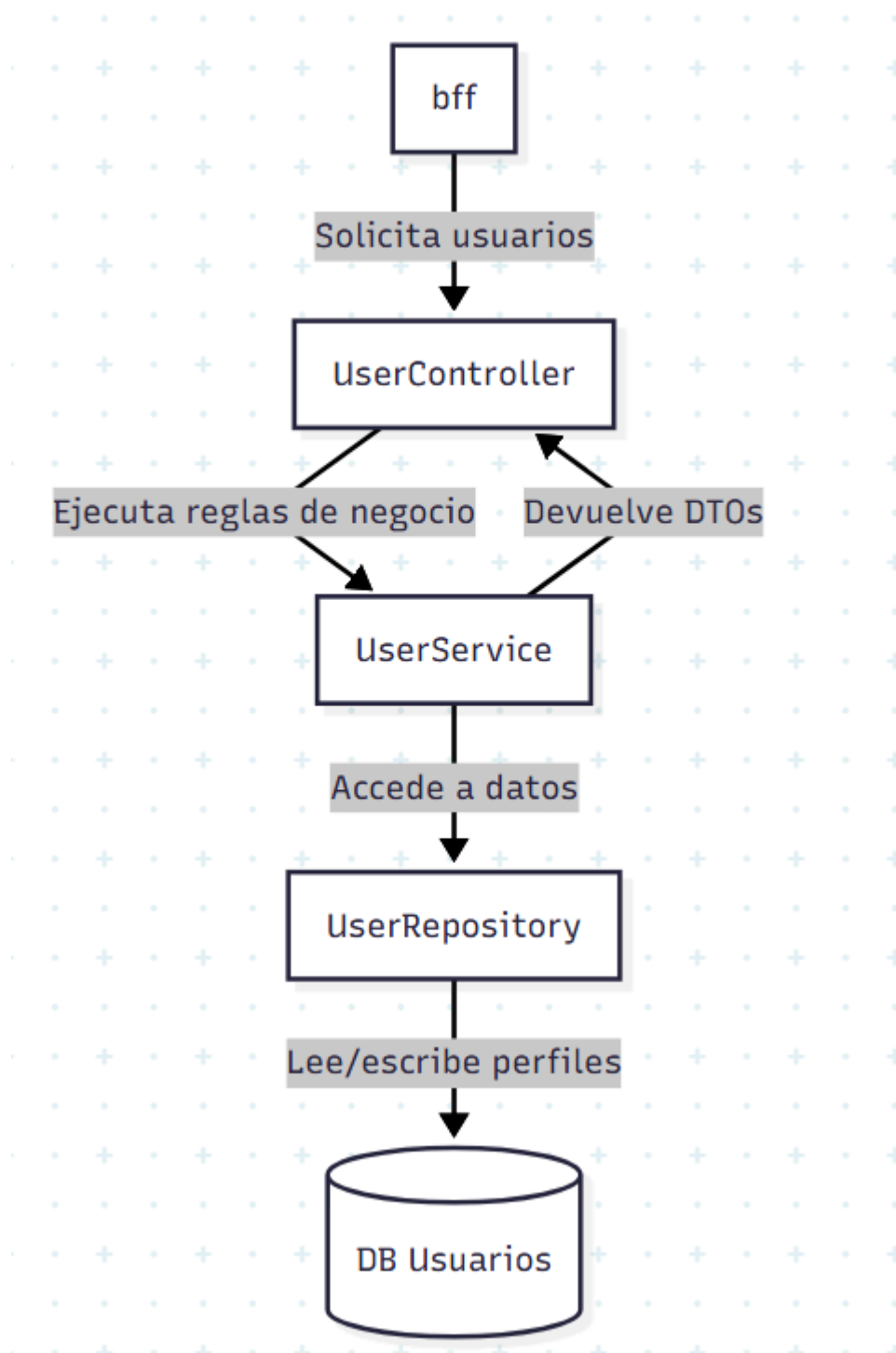


[diagrama completo \(para mejor visualización\)](#)

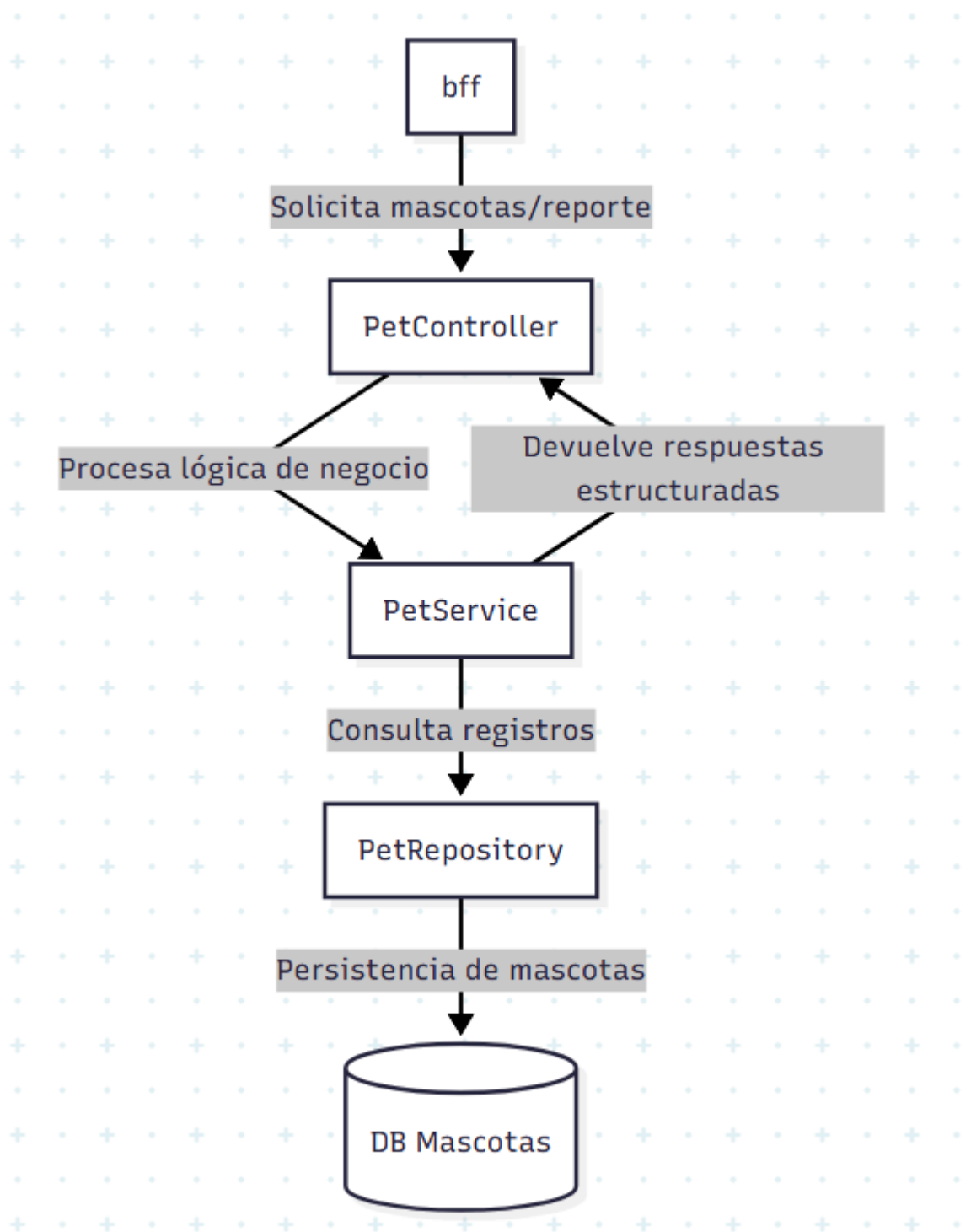
3. Diagrama Arquitectura (ms-Auth)



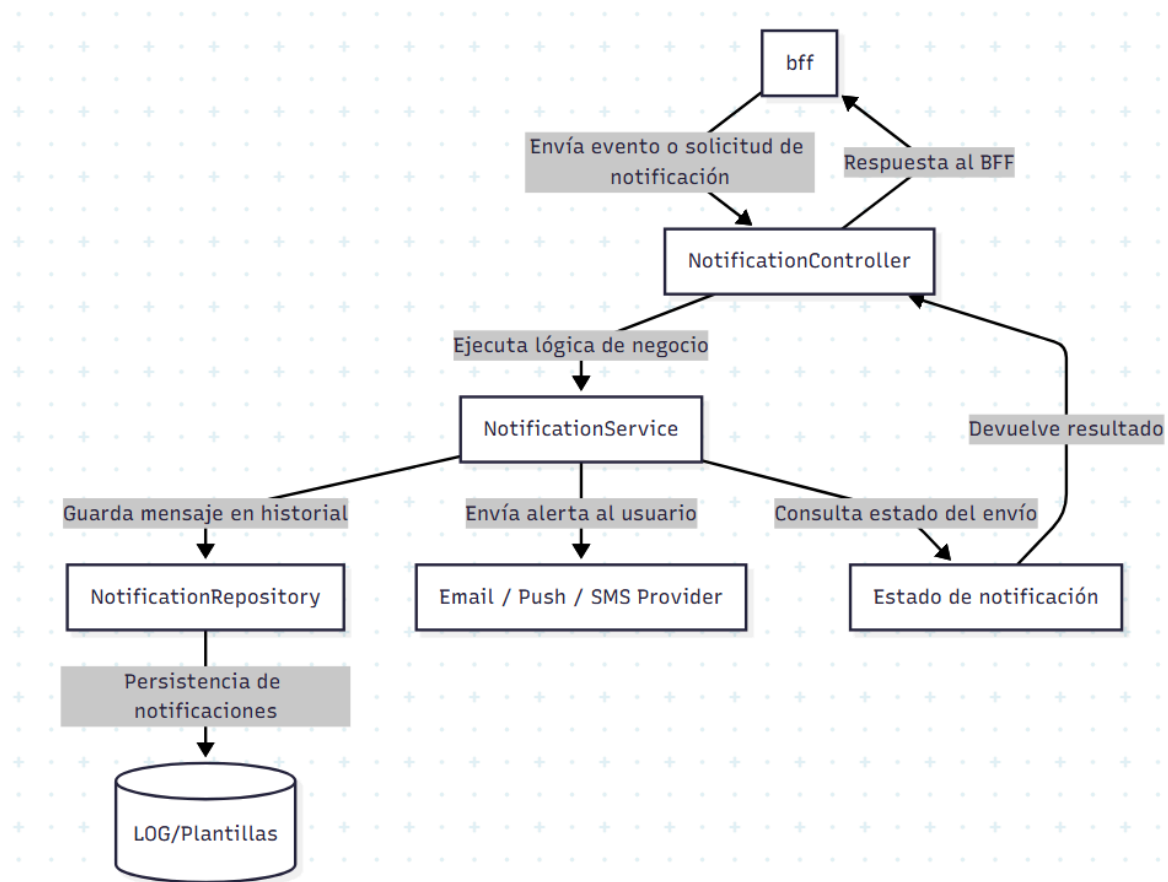
4. Diagrama Arquitectura (ms-User)



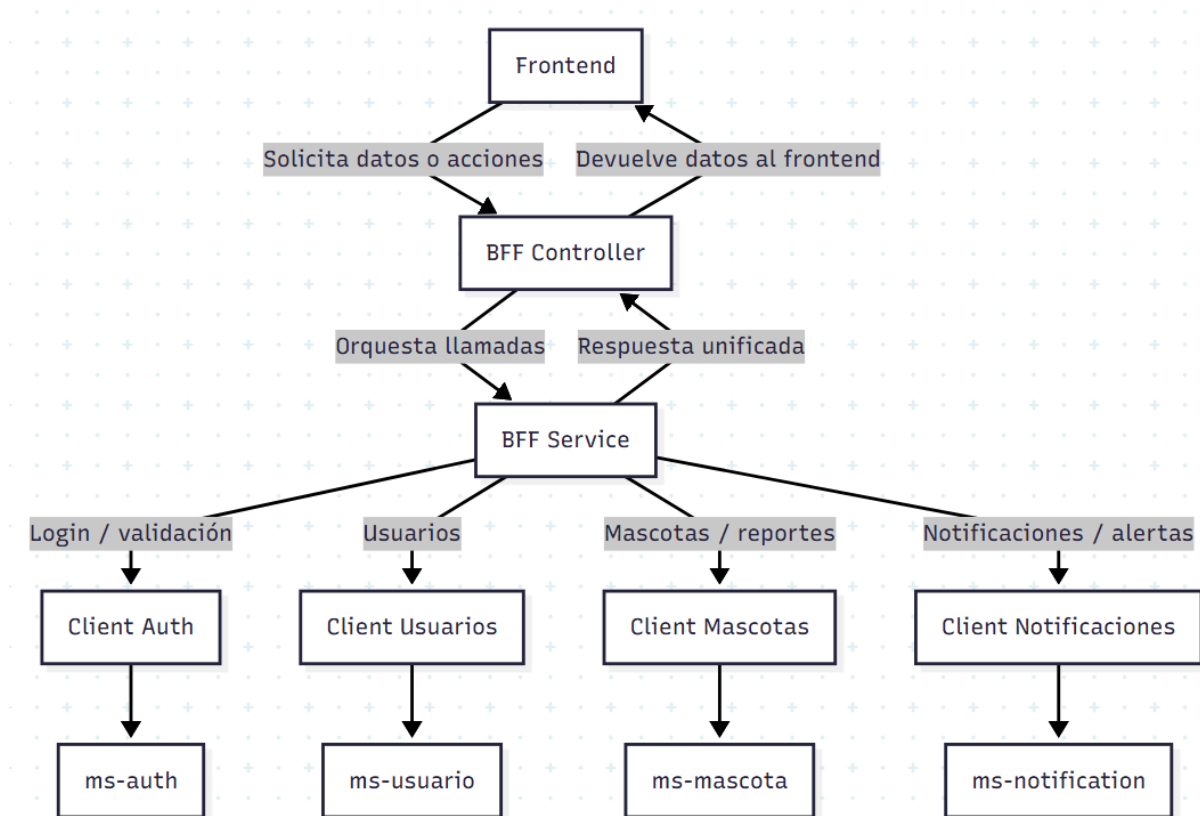
5. Diagrama Arquitectura (ms-Pet)



6. Diagrama Arquitectura (ms-Notification)



7. Diagrama Arquitectura (BFF)



8. Diagrama Arquitectura (Api Gateway)

